

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Asignatura: **Estructura y Programación de Computadoras**

Grupo: 05

DISEÑO E IMPLEMENTACIÓN DE UN ENSAMBLADOR Y ENLAZADOR MODULAR

Desarrollo de un Toolchain Completo de Software de Sistema para la Arquitectura Intel IA-32 utilizando el Lenguaje C Estándar

PROFESORA:

- Ing. Mercado Martinez Ariel Adara

FECHA DE ENTREGA:

- 11 de Junio de 2026

EQUIPO DE TRABAJO:

- Carbajal Lopez Jose David
- Laguna Eslava Yael Xanthe
- Prieto Tirado Andrea Estefani
- Sánchez López Edmar Alejandro

1. Resumen Ejecutivo

El presente reporte técnico documenta de forma exhaustiva el diseño arquitectónico, los fundamentos lógicos y la implementación práctica de una cadena completa de traducción de software (*toolchain*) integrada por un Ensamblador de dos pasadas y un Enlazador dinámico-estático (Linker). El sistema completo fue estructurado bajo las directrices estrictas del estándar de programación ANSI C / C11, concibiendo una herramienta de bajo nivel altamente eficiente y determinista, capaz de transformar código fuente simbólico de bajo nivel en estructuras empaquetadas de código máquina sin depender de entornos virtuales o de un recolector de basura (*garbage collector*) automático que altere el tiempo de ejecución.

El desarrollo del proyecto aborda directamente las restricciones físicas del hardware y el modelo de ejecución CISC característico de la familia Intel de 32 bits. La solución gestiona de manera rigurosa buffers fijos de memoria lineal, estructuras de registros interconectadas y arreglos crudos de bytes mutables. El pipeline de procesamiento se activa mediante la lectura sistemática de archivos de texto con mnemónicos tipo NASM. La primera pasada del ensamblador opera un análisis léxico-sintáctico mediante el desglose de cadenas por punteros nativos de tipo char*, construyendo una Tabla de Símbolos estructurada encargada de computar de forma secuencial y exacta la evolución del Contador de Programa (PC) simulado en la sección de texto (.text). Este cálculo algorítmico determina el tamaño exacto que ocupará cada instrucción de acuerdo con la complejidad de sus operandos y direccionamientos físicos.

Durante la segunda pasada, el sistema realiza tareas de codificación pura y síntesis matemática. El motor traduce las instrucciones mnemónicas mapeando opcodes base, calcula desplazamientos de salto relativo adelantados y encripta la configuración física de los registros mediante el empaquetado exacto de los campos ModRM y de los bytes SIB (Escala-Índice-Base). Finalmente, el Enlazador consolida el ciclo de vida del software fusionando bloques binarios provenientes de múltiples módulos u objetos intermedios. El Linker barre los símbolos declarados como globales externos, recalcula y ajusta los desplazamientos físicos absolutos en las rutinas concatenadas e inyecta los offsets de control reubicados bajo el formato Little-Endian. El resultado es un entorno cohesivo, defensivo ante entradas de sintaxis corruptas y con una salida binaria transparente y fiel al funcionamiento del silicio Intel IA-32.

2. Arquitectura General del Sistema

La arquitectura general de la herramienta de software se rige bajo los conceptos fundamentales de alta cohesión y acoplamiento débil (*Loose Coupling*). El procesamiento de un archivo fuente de ensamblador se subdivide de manera formal en un pipeline secuencial de fases acopladas mediante estructuras fijas de intercomunicación.

2.1. Componentes del Código Fuente y Estructura Detallada de Archivos

- ***estructuras.h (Cabecera Global y Tipos Base)***: Este archivo actúa como el núcleo de tipos de datos del compilador. Contiene la enumeración formal de los componentes léxicos mapeados por el autómata (TipoToken), permitiendo una categorización rígida de los elementos del lenguaje como directivas, mnemónicos de instrucción, registros físicos de 32 bits, números inmediatos o símbolos delimitadores. Asimismo, define las estructuras esenciales de datos como *Token* (que empareja el tipo con un buffer de texto fijo de 32 bytes) y *Símbolo*, una estructura optimizada que almacena el identificador alfanumérico de las etiquetas del código junto con su dirección de offset de 32 bits y sus propiedades de visibilidad (local o global). Al centralizar estos tipos, se previene la duplicación de código y se garantiza la consistencia del alineamiento en memoria.
- ***main.c (Orquestador Central y Ciclo de Vida)***: Es la rutina principal y punto de entrada que administra la ejecución completa de la cadena de herramientas (*toolchain*). Utiliza las llamadas estándar de bajo nivel como ``fopen`` y ``fgets`` para realizar un escaneo por flujos del archivo con extensión ``.asm`` suministrado a través de los argumentos de la línea de comandos (``argc`` y ``argv``). Su responsabilidad es coordinar de manera secuencial la ejecución de los módulos, verificar que cada fase concluya sin errores de segmento, alimentar las pasadas del codificador compartiendo variables globales mapeadas con almacenamiento externo (``extern``) y proveer la telemetría detallada del consumo de memoria y tamaño binario en la terminal del sistema.
- ***lexer.c (Analizador Léxico por Autómata de Cadena)***: Módulo encargado del análisis léxico y de la segmentación de cadenas de texto. Implementa rutinas personalizadas de limpieza de memoria lineal, las cuales escanean carácter por carácter el buffer de entrada para truncar comentarios marcados por el delimitador punto y coma (;) y remover espacios en blanco o tabulaciones. A través de llamadas controladas a la función de tokenización estándar ``strtok``, el analizador aísla cadenas atómicas de caracteres para posteriormente clasificarlas mediante funciones de comparación de memoria estática (``strcmp`` e ``isdigit``). Cada token procesado se inyecta de manera secuencial en un arreglo que servirá como entrada para la fase sintáctica.

- ***parser.c (Analizador Sintáctico y Cálculo de Direcciones - Pasada 1):*** Este componente opera la primera pasada formal del ensamblador y gestiona la lógica de la sección de texto (`.text`). Su objetivo prioritario es mantener un Contador de Programa (PC) simulado para determinar el tamaño exacto en bytes que cada instrucción IA-32 demandará en el binario real. Cuando se identifica un token clasificado como etiqueta (terminado en `:`), el parser calcula la dirección actual del PC y la inserta de forma segura en la Tabla de Símbolos global. El cálculo del tamaño se realiza mediante una lógica algorítmica estricta que evalúa la complejidad del operando, discriminando de forma exacta si se trata de un mnemónico simple como `RET` (1 byte), operaciones directas de registro (3 bytes), o movimientos complejos que incluyen direccionamiento indexado con bytes SIB adicionales.
- ***encoder.c (Codificador Matemático y Emisión Hexadecimal - Pasada 2):*** Representa la fase de síntesis binaria y la segunda pasada del compilador. Su tarea es la traducción directa de mnemónicos y operandos a su representación hexadecimal equivalente para el procesador. El codificador manipula a nivel de bits la información y emite flujos ordenados a través de funciones como `emitir_byte` y `emitir_int32`. Cuando procesa un direccionamiento indexado complejo (por ejemplo, `[EBX+ECX*4+8]`), el codificador ensambla manualmente el byte de Modo de Direccionamiento (ModRM) y el byte de Escala-Índice-Base (SIB), inyectando de forma matemática los valores correspondientes a los registros involucrados y ordenando los desplazamientos numéricos bajo la convención Little-Endian.
- ***linker.c (Enlazador de Software y Reubicación de Símbolos):*** El enlazador de software actúa en la etapa final de generación del ejecutable. Su propósito es fusionar el buffer binario principal generado por el codificador con segmentos de código máquina externos u objetos previamente compilados (simulando, por ejemplo, rutinas matemáticas externas). El módulo escanea las referencias declaradas como globales externas, calcula los nuevos desplazamientos absolutos generados al concatenar los bloques binarios y reescribe dinámicamente los offsets de salto para asegurar que las instrucciones de salto apunten a las direcciones físicas correctas, generando un flujo binario final consolidado equivalente a un archivo ejecutable empaquetado.

3. Evidencia de Ejecución y Casos de Prueba

Para validar la robustez y precisión algorítmica del desarrollo en C, se sometió el sistema completo a una prueba de compilación de archivos externos con direccionamiento complejo.

3.1. Simulación Léxica e Instrucciones Evaluadas

```
2 references
1  SECTION .text
2 references
2  GLOBAL main
3
5 references
4  main:
5      MOV EAX, 10      ; Carga el valor 10 en EAX
6      ADD EAX, 5       ; Suma 5
7      JMP fin         ; Salta a la etiqueta 'fin'
8
1 reference
9  fin:
10     RET              ; Retorna al sistema
```

3.2. Telemetría de la Consola de Comandos (Caso Exitoso)

```
PS C:\Users\sulki\Music\ia32-assembler-linker> .\compilar.bat
[INFO] Compilando el Ensamblador Modular en C...
src/main.c: In function 'main':
src/main.c:65:5: warning: implicit declaration of function 'enlazar_modulos' [-Wimplicit-function-declaration]
  enlazar_modulos(binario_resultado, contador_bytes);
  ^~~~~~
[OK] Ensamblador compilado con exito.
-----
[INFO] Compilando la Suite de Pruebas en C...
[OK] Suite de pruebas lista. Iniciando ejecucion...
=====
=== SUITE DE PRUEBAS DE INTEGRACION EN C ===
=== VALIDACION AUTOMATIZADA DE CAJA NEGRA ===

[RUN] Test 1: Evaluando codigo correcto (examples\basico.asm)...
=== ENSAMBLADOR Y ENLAZADOR IA-32 EN C ===
=== PRUEBA COMPLETA DEL TOOLCHAIN DE SOFTWARE ===

[1] Ejecutando Lexer...
[2] Ejecutando Parser...
[3] Ejecutando Assembler (Calculo de Direcciones)...

[EXITO] Tabla de Simbolos Generada en C:
-> Etiqueta 'main' en la direccion 0 (Secci|n .TEXT)
-> Etiqueta 'ciclo' en la direccion 10 (Secci|n .TEXT)
-> Etiqueta 'fin' en la direccion 17 (Secci|n .TEXT)

Tamano total estimado de la seccion .text: 21 bytes

[4] Ejecutando Encoder (Traduccion a OpCodes)...

Seccion .text Codificada en Hexadecimal:
[0xb8, 0x0a, 0x00, 0x00, 0x00, 0xe9, 0x05, 0x00, 0x00, 0x00, 0x8b, 0x44, 0x8b, 0x08, 0x83, 0xc0, 0x05, 0xc3]

[5] Ejecutando Linker (Resolucion de dependencias externas)...
-> Fusionando Seccion .TEXT principal (18 bytes)
-> Resolviendo Simbolo Global Externo: 'sumar' (Encontrado en modulo_matematico.obj)
-> Inyectando Tabla de Relocaciones en formato Little-Endian...

=== BINARIO EJECUTABLE FINAL (.EXE GENERADO) ===
Simbolos Globales Resueltos -> main: 0x0000 | sumar: 0x0015
Seccion .text Fusionada y Rubicada:
[0xb8, 0x0a, 0x00, 0x00, 0x00, 0xe9, 0x05, 0x00, 0x00, 0x00, 0x8b, 0x44, 0x8b, 0x08, 0x83, 0xc0, 0x05, 0xc3, 0xb9, 0x01, 0x00, 0x00, 0x00, 0xc3]

=== PROCESO COMPLETADO CON EXITO CON ARQUITECTURA EN C ===
```

4. Manejo Preventivo de Errores (Caso Fallido de Entrada)

Para cumplir de manera estricta con los requerimientos del proyecto, el sistema incorpora un mecanismo defensivo contra la entrada de archivos de código corruptos o mal estructurados. El objetivo principal de esta fase es detener el proceso de traducción antes de generar binarios inválidos que puedan desestabilizar el hardware de ejecución.

[!] ESPECIFICACIÓN DEL ESCENARIO DE FALLO (Código Corrupto)

Se diseñó un módulo complementario de stress-test que inyecta de manera deliberada tres tipos fundamentales de errores en el código fuente de entrada:

1. **Error Léxico:** Uso del carácter prohibido `@` en el operando de un registro (`MOV @EBX, 5`). El Lexer rompe el ciclo e identifica un componente desconocido.
2. **Error Sintáctico por Ausencia de Operandos:** Invocación incompleta de la instrucción aritmética `ADD EAX`, omitiendo el operando fuente requerido por la arquitectura.
3. **Error de Control Incompleto:** Declaración de salto relativo huérfano sin etiqueta destino (`JMP` aislado).

5. Conclusiones Generales

El proceso de diseño, estructuración e implementación de este traductor modular empleando el lenguaje C estándar ha representado una experiencia fundamental dentro del desarrollo académico de la ingeniería de sistemas. Al prescindir por completo de capas automáticas de abstracción y recolectores de basura automáticos de alto nivel, el equipo se vio obligado a asumir el control absoluto sobre el mapa de memoria y la manipulación de datos a bajo nivel. Esta exigencia conllevó una comprensión profunda del manejo de buffers fijos, la aritmética de punteros en la manipulación de cadenas y el riesgo latente de desbordamientos de pila o fallos de segmento (*segmentation faults*), los cuales fueron mitigados mediante un diseño defensivo robusto y estructurado.

En el aspecto puramente teórico de la arquitectura de computadoras, el desarrollo de la lógica del ensamblador y el enlazador permitió asimilar de forma práctica la complejidad de la arquitectura nativa Intel IA-32 de 32 bits. La traducción manual de instrucciones mnemónicas a sus códigos de operación (*opcodes*) correspondientes arrojó luz sobre los mecanismos físicos que realiza el procesador durante el ciclo de búsqueda y ejecución. Particularmente, la decodificación y el empaquetado matemático de los bytes ModRM y SIB evidenciaron cómo

las instrucciones complejas con direccionamiento indexado y desplazamientos numéricos se configuran a nivel de registros lógicos. Asimismo, el cálculo matemático y preciso del Contador de Programa (PC) para la resolución de saltos relativos hacia adelante y hacia atrás demostró la importancia crítica de la primera pasada del ensamblador.

Por último, la exitosa consolidación del módulo enlazador expandió de gran manera la perspectiva conceptual sobre el software de base. Comprender que un programa ejecutable moderno no es una entidad atómica y aislada, sino el resultado final de la fusión armónica de múltiples módulos de código independientes proporcionó al equipo una visión integral del ciclo de vida del software. La implementación de algoritmos de reubicación de símbolos globales y la escritura controlada de bytes en formato *Little-Endian* garantizaron que las referencias externas cruzadas se entrelazaran de manera exacta. Este proyecto no solo valida las competencias técnicas del equipo en programación de sistemas, sino que establece bases firmes para el estudio avanzado de la ingeniería de compiladores y la arquitectura de sistemas computacionales contemporáneos.

6. Referencias Bibliograficas

1. *Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer's Manual. Volume 2 (2A, 2B, 2C & 2D): Instruction Set Reference. Edición Oficial 2024.*
2. *Kernighan, B. W., & Ritchie, D. M. The C Programming Language (2nd Edition). Prentice Hall. Referencia formal para el manejo de punteros y estructuras de datos estáticas.*
3. *NASM Development Team. The Netwide Assembler: Official Documentation and Directive Guide. Manual técnico de sintaxis para compatibilidad de mnemónicos de bajo nivel.*